

CENTRO UNIVERSITÁRIO UNIRUY
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DISCIPLINA: APLICAÇÃO DE CLOUD, IOT E INDÚSTRIA 4.0 EM PYTHON
PROFESSOR: MSC HELENO CARDOSO

Aderval Santiago Leite

ECOSENSE IOT:
MONITORAMENTO AMBIENTAL COM ESP32, MQTT, API PYTHON E
DASHBOARD EM TEMPO REAL

SALVADOR - BA

2026

CENTRO UNIVERSITÁRIO UNIRUY
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DISCIPLINA: APLICAÇÃO DE CLOUD, IOT E INDÚSTRIA 4.0 EM PYTHON
PROFESSOR: MSC HELENO CARDOSO

Aderval Santiago Leite

ECOSENSE IOT:
MONITORAMENTO AMBIENTAL COM ESP32, MQTT, API PYTHON E
DASHBOARD EM TEMPO REAL

Projeto de extensão apresentado à disciplina Aplicação de Cloud, IoT e Indústria 4.0 em Python, do curso de Análise e Desenvolvimento de Sistemas, do Centro Universitário Uniruy, como requisito avaliativo parcial.

Professor: MSc Heleno Cardoso.

SALVADOR - BA

2026

RESUMO

A Internet das Coisas tem se tornado uma tecnologia cada vez mais presente em ambientes residenciais, industriais, urbanos e acadêmicos, permitindo a coleta, transmissão, armazenamento e análise de dados em tempo real. Nesse contexto, este projeto apresenta o desenvolvimento do EcoSense IoT, um sistema de monitoramento ambiental baseado em ESP32, sensores simulados, comunicação MQTT, API em Python e dashboard em tempo real. A proposta consiste em simular uma rede de sensores ambientais distribuídos em ambientes como sala de aula, laboratório, sala técnica e almoxarifado. O microcontrolador ESP32, executado na plataforma Wokwi, publica leituras de temperatura, umidade, luminosidade e CO2 em um broker MQTT público HiveMQ. Uma ponte desenvolvida em Python recebe essas mensagens e encaminha os dados para uma API FastAPI, responsável por validar, classificar e armazenar as leituras em banco SQLite. Os dados são tratados com Pandas e exibidos em um dashboard desenvolvido com Streamlit e Plotly, contendo visão geral, sensores ativos, alertas, histórico e diagnóstico do sistema. A solução também utiliza LEDs e buzzer no circuito simulado para indicar estados normais e situações de alerta. Como resultado, o projeto demonstra de forma prática a integração entre microcontroladores, comunicação IoT, backend, armazenamento, análise de dados e visualização em tempo real, contribuindo para o entendimento dos conceitos de IoT, Cloud, Indústria 4.0 e análise de dados com Python.

Palavras-chave: Internet das Coisas. ESP32. MQTT. FastAPI. Pandas. Dashboard. Indústria 4.0.

SUMÁRIO

1 INTRODUÇÃO

2 JUSTIFICATIVA

3 PROBLEMA

4 OBJETIVOS

4.1 Objetivo geral

4.2 Objetivos específicos

5 FUNDAMENTAÇÃO TEÓRICA

5.1 Internet das Coisas

5.2 Microcontroladores

5.3 Sensores ambientais

5.4 Comunicação IoT via MQTT

5.5 API Python

5.6 Banco de dados SQLite

5.7 Pandas na análise de dados

5.8 Dashboard em tempo real

5.9 Indústria 4.0

6 METODOLOGIA

7 DESENVOLVIMENTO DO PROJETO

7.1 Visão geral da solução

7.2 Arquitetura do sistema

7.3 Simulação IoT com ESP32 e Wokwi

7.4 Comunicação MQTT e ponte Python

7.5 API para recebimento dos dados

7.6 Armazenamento das leituras

7.7 Análise dos dados com Pandas

7.8 Dashboard em tempo real

7.9 Regras de alerta

8 COMPONENTES UTILIZADOS

9 FUNCIONAMENTO DO SISTEMA

10 RESULTADOS ALCANÇADOS E ESPERADOS

11 CRONOGRAMA

12 PLANO DE APRESENTAÇÃO

13 ORGANIZAÇÃO DO REPOSITÓRIO GITHUB

14 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

APÊNDICE A - ARQUIVOS DO MICROCONTROLADOR

APÊNDICE B - API E PONTE MQTT

APÊNDICE C - DASHBOARD

APÊNDICE D - INSTRUÇÕES DE EXECUÇÃO

APÊNDICE E - ESTRUTURA DO REPOSITÓRIO

1 INTRODUÇÃO

A evolução tecnológica tem transformado a forma como pessoas, empresas, cidades e indústrias coletam e utilizam informações. Nesse cenário, a Internet das Coisas, conhecida como IoT, destaca-se por permitir que dispositivos físicos sejam conectados a redes de comunicação para coletar, processar, transmitir e analisar dados em tempo real.

A IoT está presente em diversas áreas, como casas inteligentes, cidades inteligentes, agricultura, saúde, logística, segurança, automação predial e indústria. Por meio de sensores, microcontroladores e sistemas computacionais, é possível monitorar ambientes, detectar alterações, gerar alertas e auxiliar na tomada de decisão.

No contexto da Indústria 4.0, a integração entre dispositivos físicos, sistemas digitais, armazenamento de dados e análise em tempo real é essencial para tornar processos mais inteligentes, automatizados e eficientes. Assim, projetos que unem microcontroladores, sensores, comunicação em rede, banco de dados, análise de dados e visualização gráfica são fundamentais para compreender a aplicação prática dessas tecnologias.

Este projeto apresenta o desenvolvimento do EcoSense IoT, um sistema de monitoramento ambiental com ESP32, MQTT, API Python, banco SQLite e dashboard em tempo real. O sistema simula múltiplos sensores ambientais na plataforma Wokwi, envia os dados por MQTT para um broker HiveMQ, processa as mensagens por meio de uma ponte Python, registra as leituras em uma API FastAPI e exibe as informações em um dashboard desenvolvido com Streamlit e Plotly.

2 JUSTIFICATIVA

Ambientes residenciais, acadêmicos e industriais podem sofrer impactos negativos quando não há monitoramento adequado de variáveis ambientais. Temperaturas elevadas, umidade excessiva, baixa luminosidade ou concentração elevada de gases podem comprometer o conforto, a segurança, a conservação de equipamentos e até a saúde das pessoas.

Em salas de aula, laboratórios, estoques, ambientes industriais e salas técnicas, o acompanhamento dessas variáveis pode auxiliar na prevenção de falhas, na melhoria da qualidade do ambiente e no aumento da segurança. Por exemplo, o aumento da temperatura em uma sala técnica pode indicar risco para equipamentos; a umidade elevada pode prejudicar materiais armazenados; e altos níveis de CO₂ podem indicar baixa qualidade do ar.

A escolha do tema justifica-se pela relevância da IoT na sociedade atual e pela possibilidade de demonstrar, em um único projeto, conceitos importantes da disciplina, como microcontroladores, sensores, comunicação MQTT, backend em Python, armazenamento, análise com Pandas e visualização em dashboard.

3 PROBLEMA

A ausência de monitoramento contínuo em ambientes internos pode dificultar a identificação de situações críticas, como temperatura elevada, umidade inadequada, baixa luminosidade ou aumento do nível de CO₂. Em muitos casos, essas alterações somente são percebidas quando já causaram desconforto, prejuízo ou risco operacional.

Diante disso, surge a seguinte questão-problema: como desenvolver uma solução IoT capaz de simular uma rede de sensores ambientais, transmitir essas informações por um protocolo adequado ao contexto de IoT, registrar os dados em uma aplicação Python e exibi-los em tempo real em um dashboard para auxiliar na identificação de anomalias?

4 OBJETIVOS

4.1 Objetivo geral

Desenvolver um sistema IoT de monitoramento ambiental utilizando ESP32, sensores simulados, comunicação MQTT, API Python, banco de dados SQLite, análise com Pandas e dashboard em tempo real.

4.2 Objetivos específicos

- Simular uma rede IoT com microcontrolador ESP32 na plataforma Wokwi;
- Simular múltiplos sensores ambientais para sala de aula, laboratório, sala técnica e almoxarifado;
- Coletar dados ambientais de temperatura, umidade, luminosidade e CO₂ simulado;
- Implementar comunicação MQTT entre o ESP32 e um broker público HiveMQ;
- Desenvolver uma ponte Python para receber mensagens MQTT e encaminhar os dados para a API;
- Desenvolver uma API com FastAPI para receber, classificar e registrar as leituras dos sensores;

- Armazenar os dados em banco SQLite;
- Utilizar Pandas para tratamento, organização e análise dos dados coletados;
- Construir um dashboard em tempo real com Streamlit e Plotly;
- Exibir métricas, gráficos, sensores ativos, histórico, diagnóstico e alertas no dashboard;
- Documentar o projeto em repositório GitHub, incluindo código, imagens do circuito, documentação e apresentação.

5 FUNDAMENTAÇÃO TEÓRICA

5.1 Internet das Coisas

A Internet das Coisas, ou IoT, refere-se à conexão de dispositivos físicos a redes de comunicação, permitindo que esses dispositivos colem, transmitam e recebam dados. Esses dispositivos podem incluir sensores, atuadores, microcontroladores, máquinas, veículos, equipamentos domésticos e sistemas industriais.

Em uma solução IoT, os sensores são responsáveis por coletar informações do ambiente físico. Essas informações podem representar temperatura, umidade, presença, luminosidade, pressão, gases, localização, vibração, entre outros dados. Após a coleta, os dados são processados por um microcontrolador ou gateway e enviados para um servidor, API, banco de dados ou plataforma em nuvem.

5.2 Microcontroladores

Microcontroladores são pequenos dispositivos eletrônicos programáveis capazes de controlar sensores, atuadores e outros componentes. Eles são utilizados em sistemas embarcados para executar tarefas específicas, como leitura de sensores, acionamento de LEDs, controle de motores, comunicação com redes e envio de dados.

Neste projeto, o microcontrolador escolhido foi o ESP32. Ele é amplamente utilizado em aplicações IoT por possuir recursos como conectividade Wi-Fi, entradas e saídas digitais, entradas analógicas, baixo custo e compatibilidade com diferentes sensores.

5.3 Sensores ambientais

Sensores ambientais são dispositivos utilizados para medir condições físicas ou químicas do ambiente. Entre os sensores mais comuns em projetos IoT estão sensores de temperatura, umidade, luminosidade, pressão, presença e qualidade do ar.

Neste projeto, o sensor DHT22 é utilizado na simulação para leitura de temperatura e umidade. Também são utilizados potenciômetros para representar variações de CO₂ e luminosidade. Além disso, o circuito utiliza LEDs e buzzer para sinalização visual e sonora de alertas.

5.4 Comunicação IoT via MQTT

A comunicação é uma das etapas fundamentais de uma solução IoT. Após a leitura dos sensores, o microcontrolador precisa transmitir os dados para uma aplicação responsável por armazenar, processar e exibir essas informações.

Entre os protocolos utilizados em IoT, destaca-se o MQTT, um protocolo leve baseado no modelo de publicação e assinatura. Nesse modelo, dispositivos publicam mensagens em tópicos específicos e clientes assinantes recebem essas mensagens por meio de um broker. Essa abordagem é adequada para projetos IoT por ser simples, leve e eficiente.

Na versão final do EcoSense IoT, o ESP32 publica as leituras no tópico `ecosense/santiago/leituras` em um broker público HiveMQ. Uma ponte Python, implementada com a biblioteca Paho MQTT, assina esse tópico, recebe as mensagens e as encaminha para a API FastAPI local.

5.5 API Python

Uma API, ou Interface de Programação de Aplicações, permite a comunicação entre sistemas. No contexto deste projeto, a API é responsável por receber os dados encaminhados pela ponte MQTT, validar as informações, definir o status da leitura e armazenar os dados no banco.

A API foi desenvolvida com FastAPI, um framework moderno da linguagem Python voltado para criação de aplicações web e serviços REST. A escolha do FastAPI se justifica pela simplicidade de implementação, boa performance, documentação automática e facilidade de integração com outros sistemas.

5.6 Banco de dados SQLite

O armazenamento dos dados é uma etapa importante em sistemas IoT, pois permite registrar o histórico das leituras e realizar análises posteriores. Neste projeto, foi utilizado o banco de dados SQLite, por ser leve, simples, local e adequado para protótipos acadêmicos.

5.7 Pandas na análise de dados

Pandas é uma biblioteca da linguagem Python utilizada para manipulação e análise de dados. No EcoSense IoT, o Pandas é utilizado no dashboard para tratar os dados retornados pela API, realizar filtragens, identificar sensores ativos, calcular indicadores e preparar os dados para os gráficos.

5.8 Dashboard em tempo real

Dashboard é uma interface visual utilizada para apresentar informações de forma organizada, facilitando a interpretação dos dados. Em sistemas IoT, dashboards são importantes porque permitem acompanhar as leituras dos sensores, verificar o estado dos dispositivos, analisar tendências e identificar situações críticas.

O dashboard do EcoSense IoT foi desenvolvido com Streamlit e Plotly, apresentando páginas de visão geral, sensores ativos, alertas, histórico e diagnóstico. A interface atualiza automaticamente, permitindo acompanhar as leituras praticamente em tempo real.

5.9 Indústria 4.0

A Indústria 4.0 representa a integração entre tecnologias digitais, automação, conectividade, análise de dados e sistemas inteligentes. Nesse contexto, sensores e microcontroladores permitem monitorar máquinas, ambientes e processos em tempo real. O EcoSense IoT se relaciona com esse conceito ao demonstrar como sensores conectados podem gerar dados úteis para análise, alerta e visualização.

6 METODOLOGIA

A metodologia utilizada neste projeto é de natureza aplicada, pois busca desenvolver uma solução prática para monitoramento ambiental utilizando conceitos de IoT, programação Python e visualização de dados. Quanto aos procedimentos, o projeto foi desenvolvido por meio de simulação computacional e implementação de software.

A simulação do circuito foi realizada na plataforma Wokwi, utilizando o microcontrolador ESP32 e componentes virtuais. A aplicação de software foi desenvolvida em ambiente local com VS Code, utilizando Python, FastAPI, SQLite, Pandas, Streamlit, Plotly e Paho MQTT.

- Definição do problema e da proposta de solução;
- Escolha dos componentes eletrônicos e tecnologias de software;
- Montagem do circuito simulado no Wokwi;
- Programação do ESP32 para simular múltiplos sensores e publicar dados via MQTT;
- Desenvolvimento da ponte MQTT em Python;
- Desenvolvimento da API em Python para recebimento das leituras;
- Criação do banco de dados SQLite para armazenamento;
- Desenvolvimento do dashboard para visualização em tempo real;
- Testes de funcionamento com condições normais e situações de alerta;
- Organização do repositório GitHub, documentação e apresentação.

7 DESENVOLVIMENTO DO PROJETO

7.1 Visão geral da solução

O EcoSense IoT foi projetado como uma solução integrada de monitoramento ambiental. O sistema começa na camada de simulação IoT, representada pelo circuito no Wokwi. Nessa camada, o ESP32 atua como um gateway que simula leituras de múltiplos sensores ambientais.

Após a geração das leituras, o ESP32 publica mensagens em formato JSON no broker MQTT. A ponte Python recebe essas mensagens, valida os campos essenciais e encaminha os dados para a API FastAPI. A API classifica cada leitura como NORMAL ou ALERTA e a armazena no banco SQLite. O dashboard consulta a API, transforma os dados com Pandas e apresenta as informações em gráficos e indicadores.

7.2 Arquitetura do sistema

A arquitetura final do sistema é composta por sete camadas principais: dispositivos IoT, comunicação MQTT, broker, ponte Python, backend, armazenamento, análise e visualização.

- Camada 1 - Dispositivos IoT: ESP32, DHT22, potenciômetros, LEDs e buzzer na plataforma Wokwi;

- Camada 2 - Comunicação MQTT: publicação das leituras em tópico MQTT;
- Camada 3 - Broker MQTT: HiveMQ público, responsável por intermediar as mensagens;
- Camada 4 - Ponte Python: script `mqtt_bridge.py` responsável por assinar o tópico e encaminhar dados para a API;
- Camada 5 - Backend: API FastAPI responsável por receber, validar, classificar e armazenar leituras;
- Camada 6 - Armazenamento: banco SQLite com histórico das leituras;
- Camada 7 - Visualização: dashboard Streamlit com Pandas e Plotly.

ESP32/Wokwi -> MQTT HiveMQ -> `mqtt_bridge.py` -> FastAPI -> SQLite -> Dashboard Streamlit

7.3 Simulação IoT com ESP32 e Wokwi

A simulação do circuito foi realizada na plataforma Wokwi. O microcontrolador utilizado foi o ESP32, escolhido por possuir conectividade Wi-Fi e ser adequado para aplicações IoT. O circuito conta com sensor DHT22, potenciômetros, LEDs individuais por ambiente, LED verde geral, LED vermelho geral e buzzer.

A versão final simula quatro ambientes: sala de aula, laboratório, sala técnica e almoxarifado. Cada ambiente possui um sensor lógico identificado por `sensor_id` próprio, permitindo que o dashboard trate os dados separadamente.

7.4 Comunicação MQTT e ponte Python

A comunicação entre o Wokwi e o sistema local foi implementada com MQTT, substituindo a abordagem inicial baseada em HTTP direto e túnel externo. Essa decisão tornou a solução mais adequada ao contexto de IoT e evitou instabilidades de HTTPS em ambiente simulado.

O ESP32 publica as mensagens no tópico `ecosense/santiago/leituras`. A ponte Python `mqtt_bridge.py`, utilizando a biblioteca Paho MQTT, conecta-se ao broker HiveMQ, assina o tópico e encaminha cada payload recebido para a API local executada na porta 8050.

7.5 API para recebimento dos dados

A API foi desenvolvida com FastAPI e possui rotas para receber leituras, listar registros, retornar a última leitura, gerar estatísticas e apagar dados de teste. A principal rota de entrada é POST /leituras, responsável por receber os dados em formato JSON.

7.6 Armazenamento das leituras

As leituras recebidas pela API são armazenadas em banco SQLite. A tabela principal do projeto é chamada leituras e possui os campos id, sensor_id, timestamp, temperatura, umidade, co2, luminosidade e status.

7.7 Análise dos dados com Pandas

O Pandas é utilizado no dashboard para organizar e analisar os dados coletados. Entre as operações realizadas estão conversão de datas, conversão de colunas numéricas, filtragem por sensores ativos, seleção da última leitura por sensor, cálculo da taxa de alerta e preparação dos dados para gráficos.

7.8 Dashboard em tempo real

O dashboard foi desenvolvido com Streamlit e Plotly. A interface possui menu lateral com as páginas Visão Geral, Sensores Ativos, Alertas, Histórico e Diagnóstico. A tela Sensores Ativos exibe somente sensores que enviaram leituras recentemente, evitando que sensores antigos ou de teste poluam a apresentação.

7.9 Regras de alerta

A classificação do status é feita pela API. Uma leitura é considerada ALERTA quando temperatura é maior que 30°C, umidade é maior que 80%, CO2 é maior que 1000 ppm ou luminosidade é menor que 200. Caso contrário, a leitura é classificada como NORMAL.

8 COMPONENTES UTILIZADOS

Categoria	Item	Finalidade
Microcontrolador	ESP32	Simular gateway IoT, publicar dados via Wi-Fi e MQTT.
Sensor	DHT22	Representar leitura de temperatura e umidade.

Sensor simulado	Potenciômetros	Simular variação de CO2 e luminosidade.
Atuador	LED verde geral	Indicar estado geral normal.
Atuador	LED vermelho geral	Indicar existência de alerta.
Atuador	LEDs individuais	Indicar ambiente específico em alerta.
Atuador	Buzzer	Emitir bipe curto quando o sistema entra em alerta.
Simulação	Wokwi	Montagem e execução virtual do circuito.
Comunicação	MQTT / HiveMQ	Transmissão de mensagens IoT.
Backend	FastAPI e Uvicorn	Criar API para recebimento e consulta das leituras.
Banco de dados	SQLite	Armazenar histórico das leituras.
Análise	Pandas	Tratar dados e gerar indicadores.
Dashboard	Streamlit e Plotly	Exibir gráficos e informações em tempo real.

9 FUNCIONAMENTO DO SISTEMA

O funcionamento do sistema ocorre em etapas integradas. Primeiramente, o ESP32 simula leituras ambientais para quatro ambientes distintos. Para cada ambiente são gerados valores de temperatura, umidade, CO2 e luminosidade. As leituras variam ao longo do tempo, permitindo demonstrar estados normais e situações de alerta.

Após gerar os dados, o ESP32 publica uma mensagem MQTT para cada sensor no tópico configurado. O broker HiveMQ distribui a mensagem para a ponte Python, que está inscrita no tópico. A ponte converte o payload JSON, valida os campos obrigatórios e envia os dados para a API local por meio de uma requisição HTTP POST.

A API recebe os dados, calcula o status da leitura, armazena as informações no SQLite e disponibiliza os registros para consulta. O dashboard acessa a API em intervalos regulares, carrega os dados em DataFrame Pandas e atualiza os gráficos, cards, tabelas e indicadores.

10 RESULTADOS ALCANÇADOS E ESPERADOS

O projeto EcoSense IoT alcançou a integração funcional entre simulação IoT, comunicação MQTT, processamento em Python, armazenamento local e dashboard em tempo real. O sistema permite acompanhar múltiplos sensores ativos, identificar alertas e visualizar dados em gráficos.

- Funcionamento da simulação IoT no Wokwi;
- Conexão do ESP32 ao Wi-Fi simulado;
- Publicação de mensagens MQTT no broker HiveMQ;
- Recebimento das mensagens pela ponte Python;
- Envio das leituras para a API FastAPI;
- Armazenamento das leituras no banco SQLite;
- Atualização do dashboard em tempo real;
- Exibição de sensores ativos, gráficos, histórico, diagnóstico e alertas;
- Sinalização visual no circuito por LEDs gerais e individuais;
- Buzzer com bipe curto em situação de alerta.

11 CRONOGRAMA

Etapa	Descrição	Resultado
1	Definição do tema e levantamento dos requisitos.	Tema e problema definidos.
2	Planejamento da arquitetura e tecnologias.	Arquitetura técnica documentada.
3	Montagem da simulação IoT no Wokwi.	Circuito virtual funcional.
4	Desenvolvimento do código do ESP32 com MQTT.	Publicação de mensagens MQTT.

5	Desenvolvimento da ponte MQTT em Python.	Integração entre broker e API.
6	Desenvolvimento da API com FastAPI.	Endpoints funcionando.
7	Integração com banco SQLite.	Histórico armazenado.
8	Desenvolvimento do dashboard.	Gráficos e indicadores em tempo real.
9	Testes de integração.	Sistema validado de ponta a ponta.
10	Organização do GitHub, documentação e slides.	Entrega final preparada.

12 PLANO DE APRESENTAÇÃO

Tempo aproximado	Conteúdo	Responsável
1 min	Apresentação do problema e justificativa.	Aderval Santiago Leite
1 min	Objetivo geral, objetivos específicos e tecnologias utilizadas.	Aderval Santiago Leite
2 min	Explicação da arquitetura IoT com MQTT, broker, ponte Python, API, banco e dashboard.	Aderval Santiago Leite
2 min	Demonstração do circuito no Wokwi, LEDs individuais e buzzer.	Aderval Santiago Leite
2 min	Demonstração da ponte MQTT, API e dashboard recebendo dados em tempo real.	Aderval Santiago Leite
1 min	Resultados, conclusão e possibilidades futuras.	Aderval Santiago Leite

Durante a demonstração, recomenda-se deixar abertas duas janelas principais: uma com a simulação do Wokwi e outra com o dashboard. A API e a ponte MQTT devem estar rodando em terminais separados.

13 ORGANIZAÇÃO DO REPOSITÓRIO GITHUB

```
ecosense-iot/
├── README.md
├── requirements.txt
├── .gitignore
├── api/
│   ├── __init__.py
│   ├── main.py
│   ├── database.py
│   ├── schemas.py
│   ├── mqtt_bridge.py
│   └── simulador_envio.py
├── dashboard/
│   ├── __init__.py
│   ├── app.py
│   └── services.py
├── microcontrolador/
│   ├── sketch.ino
│   └── diagram.json
├── circuito/
│   ├── vista_circuito.png
│   ├── dashboard_visao_geral.png
│   ├── dashboard_sensores_ativos.png
│   └── link_wokwi.txt
├── documentacao/
│   ├── projeto_abnt.docx
│   └── projeto_abnt.pdf
└── slides/
    └── apresentacao_ecosense_iot.pptx
```

14 CONSIDERAÇÕES FINAIS

O projeto EcoSense IoT apresenta uma solução prática de monitoramento ambiental utilizando conceitos de Internet das Coisas, microcontroladores, comunicação MQTT, API Python, armazenamento de dados, análise com Pandas e visualização em dashboard.

A versão final do projeto utiliza uma arquitetura mais adequada ao contexto IoT, substituindo a tentativa inicial de comunicação HTTP direta por uma solução baseada em MQTT, broker HiveMQ e ponte Python. Essa arquitetura demonstrou maior estabilidade na simulação e melhor alinhamento com práticas comuns em aplicações de Internet das Coisas.

A utilização do ESP32 em ambiente simulado permite demonstrar o funcionamento do circuito de forma acessível e segura. A API Python, o banco SQLite e o dashboard próprio tornam o projeto mais completo, aproximando-o de uma aplicação real.

Conclui-se que o EcoSense IoT atende aos objetivos propostos, pois integra hardware simulado, comunicação IoT, backend, banco de dados, análise de dados e visualização, demonstrando uma solução funcional baseada em IoT e aplicável a diferentes contextos residenciais, acadêmicos e industriais.

REFERÊNCIAS

CARDOSO, Heleno. Projeto de Extensão 2026.1: Aplicação de Cloud, IoT e Indústria 4.0 em Python. Salvador: Wyden UniRuy, 2026.

CARDOSO, Heleno. Prática de Big Data com IoT no Google Colab. Salvador: Wyden UniRuy, 2026.

MCROBERTS, Michael. Arduino básico. São Paulo: Novatec, 2011.

MONK, Simon. Programação com Arduino: começando com sketches. Porto Alegre: Bookman, 2017.

OLIVEIRA, Sérgio de. Internet das Coisas com ESP8266, Arduino e Raspberry Pi. São Paulo: Novatec, 2017.

PYTHON SOFTWARE FOUNDATION. Python Documentation. Disponível em: <https://docs.python.org/>. Acesso em: 08 maio 2026.

PANDAS DEVELOPMENT TEAM. Pandas Documentation. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: 08 maio 2026.

FASTAPI. FastAPI Documentation. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 08 maio 2026.

STREAMLIT. Streamlit Documentation. Disponível em: <https://docs.streamlit.io/>. Acesso em: 15 maio 2026.

PLOTLY. Plotly Python Open Source Graphing Library. Disponível em: <https://plotly.com/python/>. Acesso em: 15 maio 2026.

WOKWI. Wokwi Documentation. Disponível em: <https://docs.wokwi.com/>. Acesso em: 15 maio 2026.

HIVEMQ. MQTT Essentials. Disponível em: <https://www.hivemq.com/mqtt/>. Acesso em: 15 maio 2026.

ECLIPSE FOUNDATION. Paho MQTT Python Client Documentation. Disponível em: <https://eclipse.dev/paho/>. Acesso em: 15 maio 2026.

APÊNDICE A - ARQUIVOS DO MICROCONTROLADOR

O código final do microcontrolador encontra-se no arquivo microcontrolador/sketch.ino. Ele utiliza ESP32, Wi-Fi do Wokwi, PubSubClient, DHT22, LEDs e buzzer. A comunicação com o sistema é feita por MQTT no tópico ecosense/santiago/leituras.

```
Principais recursos do sketch.ino:  
- Publicação MQTT no broker.hivemq.com, porta 1883;  
- Simulação dos sensores SALA_AULA_01, LABORATORIO_01, SALA_TECNICA_01 e  
  ALMOXARIFADO_01;  
- Envio de temperatura, umidade, CO2 e luminosidade;  
- LEDs individuais por ambiente;  
- LED verde geral para estado normal;  
- LED vermelho geral para alerta;  
- Buzzer com bipe curto ao entrar em alerta;  
- Primeiro envio imediato e ciclos periódicos de envio.
```

APÊNDICE B - API E PONTE MQTT

A integração entre o broker MQTT e a API local é realizada pelo arquivo api/mqtt_bridge.py. Esse script assina o tópico MQTT, recebe mensagens em JSON e as encaminha para a rota POST /leituras da API FastAPI.

```
Comandos principais:  
uvicorn api.main:app --reload --port 8050  
python api/mqtt_bridge.py
```

```
Tópico MQTT:  
ecosense/santiago/leituras
```

```
Rota principal da API:  
POST http://127.0.0.1:8050/leituras
```

APÊNDICE C - DASHBOARD

O dashboard final encontra-se em `dashboard/app.py`. A interface possui páginas para visão geral, sensores ativos, alertas, histórico e diagnóstico. O arquivo `dashboard/services.py` centraliza as chamadas para a API.

```
Comando para execução:  
streamlit run dashboard/app.py
```

```
Principais páginas:  
- Visão Geral;  
- Sensores Ativos;  
- Alertas;  
- Histórico;  
- Diagnóstico.
```

APÊNDICE D - INSTRUÇÕES DE EXECUÇÃO

```
# Criar ambiente virtual  
python -m venv venv  
  
# Ativar no Windows PowerShell  
.\venv\Scripts\Activate.ps1  
  
# Instalar dependências  
pip install -r requirements.txt  
  
# Rodar a API  
uvicorn api.main:app --reload --port 8050  
  
# Rodar o dashboard em outro terminal  
streamlit run dashboard/app.py  
  
# Rodar a ponte MQTT em outro terminal  
python api/mqtt_bridge.py  
  
# Opcional: rodar simulador de envio como backup  
python api/simulador_envio.py
```

APÊNDICE E - ESTRUTURA DO REPOSITÓRIO

A estrutura final recomendada para entrega no GitHub está apresentada na seção 13. Os arquivos de código, circuito, documentação, imagens e slides devem ser mantidos organizados em suas respectivas pastas.